A-EVOLVE - ARCHITECTURE & DESIGN

LAYERED ARCHITECTURE DIAGRAM

User / CLI / Script

evolver = ae.Evalver(agent="swe", benchmark="swe-verified")

API LAYER (api.py)

- _resolve_agent()
- _resolve_bench()
- _resolve_config()

ENGINE LAYER (engine)

EvolutionLoop

- Solve
- Observe
- SnapShot

engine.step()

Shared Primitives

- TrialRunner
- EvolutionHIstory
- Observer
- VersionControl

ALGORITHMS LAYER (algorithms/)

aevolve/

- engine.py
- Prompts.py
- tools.py
- Gating.py

Future Algorithm

Protocol Layer

- BaseAgent

Benchmark Layer

- SWEBench
- MCP-Atlas
- Terminal
- SkillBench

LLM Provider Layer

- Anthropic
- OpenAI
- Bedrock

Contract Layer (contract/)

AgentWorkspace (workspace.py)

- manifest.yaml
- prompts
- skiils
- memory
- tools

EVOLUTION LOOP WORKFLOW

1. SOLVE PHASE:
   Run tasks and gather feedback
2. OBSERVE PHASE:
   Collect agent observations
3. PRE-EVOLVE SNAPSHOT:
   Save pre-evolution state
4. ENGINE STEP:
   Execute evolution and validate changes
   - **Free algorithm implementation:** LLM mutate workspace, validate, history query, gating logic
5. POST-EVOLVE SNAPSHOT
   Record updated state and history
6. RELOAD PHASE
   Reload agent for next cycle

Score converged? cycle < max?

- Yes → Continue Loop (Solve Phase)
- No → Return EvolutionResult